

# Guía Tema 5 Orientación a Objetos

## Setters y Getters

Creamos una función pública con identificador de this

### Ejemplo de Setter:

```
public void setId (int id){this.id=id;}
```

### Ejemplo de Getter

```
public int getId (){return this.id;}
```

En setter básicamente asignamos el valor a un atributo

En getter obtenemos ese atributo

Esto se usa en ámbitos privados donde no queremos acceso fácil a nuestros private atributos

El this lo usamos para decir que nos referimos al atributo en cuestión de la instancia

## toString()

Método usado para hacer un return de un tipo String y poder pasar fácilmente una instancia a texto

Es importante usar el @Override antes del toString()

```
@Override
```

```
public String toString(){return "Hola " + this.nombre;}
```

## Estáticos

Usamos **static** en atributos únicos dentro de una clase

Por ejemplo un contador o un valor total de la suma de todas las instancias

```
private static int contador;
```

```
private static int recaudacionTotal;
```

## Constantes

Usamos **final** únicamente en atributos que no vamos a modificar, como podría ser el valor de un precio fijo o las filas o columnas de una matriz

## Public, Private o ninguna

Usamos **public** en métodos accesibles desde fuera de la clase, **private** en aquellos que no queremos que se usen fuera de la clase. Si no ponemos **nada** se llama package private, es decir, podemos acceder desde fuera pero únicamente dentro del paquete, esto no os importa mucho ya que siempre trabajamos desde el mismo paquete, “es public” realmente.

En atributos es la misma, pero con los getters y setters para actuar como frontera, puedes acceder a atributos privados pero bajo mis condiciones básicamente.

## New Instance y Constructores

No tiene mucho misterio, un constructor es la declaración de la clase con X atributos obligatorios. Ponemos de ejemplo que tenemos la clase Pelicula. Y tenemos un constructor donde obligamos a crear la instancia con el nombre de la película y el número de la sala donde se proyecta.

```
public class Pelicula{  
  
    private String nombre;  
    private int numSala;  
    private String horaProyeccion;  
  
    public Pelicula(String nombre, int numSala){  
        this.nombre = nombre;  
        this.numSala = numSala;  
    }  
}
```

**Luego en el main haríamos esto:**

```
Pelicula p = new Pelicula("El silencio de los corderos", 2);
```

Declararíamos una instancia de película con nombre e int

Podemos crear más constructores siempre que los parámetros requeridos sean distintos.

Es decir, otro constructor donde pedimos solo el nombre, o solo el int, u otro completos donde pedimos todos los atributos.

También podemos crear uno sin atributos donde los asignamos nosotros por defecto.

Y bueno, respecto a la creación de instancias lo mismo, podemos crear tantas queramos siempre que el nombre sea distinto. A excepción de cuando las creamos para meterlas en un ArrayList o HashMap que accederemos a ella mediante índice así que es completamente indiferente.

**Ejemplo:**

Tengo un método donde cada vez que accedo a él creo una instancia de Pelicula

```
Pelicula peli = new Pelicula();  
peliculas.add(peli);
```

Luego cuando llamemos al ArrayList películas tendrá instancias diferenciadas por su posición, así que aquí usaremos el toString() o cualquier otro método

### Para explicarme mejor

Tenemos este Array

```
ArrayList<Película> películas = new ArrayList<>();
```

En un método vamos asignando todas las instancias de películas que creamos al Array películas. Luego queremos acceder a esas instancias para lo que sea, cambiar un atributo, mostrarlo, enseñar la película entera en String, etc.

Pues básicamente con un bucle **for-each**

```
for(Película p: películas){  
    System.out.println(p.toString());  
}
```

‘p’ hace referencia a la instancia en la posición en la que se encuentra el bucle, con esa instancia podemos acceder a cualquier método

Si queremos cambiar el número de sala de todas las películas que estén en la sala 3 solo sería en el mismo bucle for-each con un if

```
if(p.getNumSala()==2){  
    p.setNumSala(3);  
}
```

Por ejemplo, vaya

## Bonus Point

Si os da error un .add en un ArrayList o HashMap y os dice “Cannot Access (la clase que sea)” es un error de mentira, no pasa nada, es como un warning.